

UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE INGENIERÍA
Departamento de Ingeniería Informática



Coord-Agent: Agente que apoya la coordinación de
actividades grupales

Integrantes: Daniel Eguiluz
Sergio Espinoza
Matías Hernández
Nicolás Sánchez
Asignatura: TAVI - Diurno
Profesor: Daniel Gacitúa

Santiago - Chile

6 de mayo de 2026

Tabla de contenidos

1. Introducción	1
2. Definición del proyecto	1
2.1. Identificación del problema u oportunidad	2
2.2. Justificación del propósito	3
2.2.1. Propuesta de valor	3
2.2.2. Modelo de ingresos propuesto	3
2.3. Planteamiento de los objetivos del proyecto	4
2.3.1. Objetivo General	4
2.3.2. Objetivos específicos	4
3. Estudio de factibilidad	5
3.1. Análisis técnico	5
3.2. Análisis económico	5
3.3. Evaluación de riesgos y limitaciones	5
4. Metodología de desarrollo	5
4.1. Selección de una metodología ágil	5
4.2. Planificación de iteraciones y definición de roles en el equipo	6
4.2.1. Plan de iteraciones	6
4.3. Diagrama de usos éticos de la IA (MyAI Compass)	9
4.4. Repositorio de control de versiones	9
5. Diseño arquitectural	9
5.1. Definición de la arquitectura de software	9
5.2. Identificación de la integración del LLM en la solución	10
5.3. Diagramas UML y especificaciones técnicas	11
6. Mini-demo funcional	12
6.1. Despliegue de una instancia mínima de un LLM	12
6.1.1. LLM utilizado y justificación	13

6.1.2. Prompt de personalización inicial	14
6.1.3. Presentación y exposición	15
7. Conclusiones	15

1. Introducción

La coordinación de equipos es una actividad fundamental en cualquier organización. Sin embargo, en la práctica este proceso se apoya casi exclusivamente en herramientas simples de comunicación que no fueron diseñadas para gestionar decisiones grupales de forma estructurada. El resultado de esto es un ciclo de coordinación confuso, ya que es dependiente de la memoria y disponibilidad de un organizador, el cual es propenso a cometer errores y/o demorarse más de la cuenta en realizar dicha organización.

Los grandes modelos del lenguaje (LLMs) presentan una oportunidad para abordar dicha problemática, su capacidad de "analizar" grandes cantidades de texto en lenguaje natural de forma rápida lo convierte en el núcleo ideal para un agente que pueda interpretar entradas ambiguas, consolidar información distribuida y proponer decisiones con su respectiva justificación de forma rápida y efectiva.

El presente informe documenta la formulación del proyecto para crear un agente de IA orientado a apoyar la toma de decisiones grupales, el cuál es desarrollado como proyecto semestral para el ramo Taller de Agentes Virtuales Inteligentes (TAVI) modalidad diurna. En esta primera entrega se abordará la definición del proyecto, un estudio de la factibilidad tanto técnica como económica, la metodología de desarrollo a implementar, el diseño arquitectural de la solución y una mini-demo con una instancia mínima de un LLM.

2. Definición del proyecto

Tal como se adelantaba, en este proyecto se busca la creación de un agente conversacional capaz de transformar conversaciones desestructuradas en decisiones grupales concretas, optimizando la coordinación de equipos mediante un análisis de disponibilidad, preferencias y restricciones.

2.1. Identificación del problema u oportunidad

En entornos organizacionales, la coordinación de actividades grupales (como reuniones, planificación de tareas o eventos de equipo) se realiza comúnmente a través de herramientas de comunicación generalistas como WhatsApp, correo electrónico o plataformas de mensajería colaborativa. Sin embargo, estas herramientas no están diseñadas para gestionar procesos de coordinación de manera estructurada.

Lo anterior genera múltiples ineficiencias operativas:

1. **Fragmentación de la información:** La comunicación es desorganizada y queda distribuida en múltiples mensajes o canales.
2. **Dificultad para consolidar disponibilidad:** Cuando una cantidad considerable de personas debe reunirse puede requerir múltiples rondas de preguntas y respuestas determinar la disponibilidad del grupo, lo que con una revisión manual puede ser costoso en términos de tiempo.
3. **Procesos iterativos y prolongados:** Toma tiempo alcanzar acuerdos, el que podría usarse en otras actividades que generen valor directos, es decir, hay un costo de oportunidad asociado.
4. **Reagendamientos frecuentes:** La falta de visibilidad global sobre restricciones de los participantes provoca reprogramaciones frecuentes.
5. **Dependencia de un coordinador central:** Habitualmente, una sola persona asume el rol de consolidar la información de manera manual, generando un cuello de botella y un costo de tiempo para dicha persona.

Como consecuencia, se invierte una cantidad significativa de tiempo en tareas de coordinación, afectando la productividad y generando costos operacionales indirectos.

Esto representa una oportunidad para desarrollar un sistema que permita estructurar y automatizar estos procesos mediante interfaces conversacionales, capaces de interpretar lenguaje natural y transformar interacciones informales en decisiones concretas.

2.2. Justificación del propósito

El proyecto tiene un propósito principalmente comercial, orientado a mejorar la eficiencia operativa en equipos de trabajo y organizaciones.

2.2.1. Propuesta de valor

La propuesta de valor se centra en:

1. **Automatización:** Automatizar la toma de decisiones grupales en base a disponibilidad y restricciones
2. **Ahorro de tiempo:** Reducir el tiempo destinado a la coordinación de reuniones y actividades.
3. **Reducción de fricción:** Disminuye la fricción en la comunicación interna.
4. **Accesibilidad:** Al ser usado en lenguaje natural no requiere una capacitación técnica por parte de los usuarios.

El sistema permite capturar información incompleta o distribuida y consolidarla en resultados concretos, generando ahorros de tiempo y mejoras en productividad.

2.2.2. Modelo de ingresos propuesto

1. Software as a service (SaaS) para equipos y organizaciones.
2. Suscripción mensual por usuario o por equipo.
3. Integración con herramientas corporativas
4. Funcionalidades premium como:
 - Optimización automática de horarios.
 - Análisis de eficiencia en reuniones.
 - Gestión avanzada de equipos y disponibilidad.

2.3. Planteamiento de los objetivos del proyecto

2.3.1. Objetivo General

Desarrollar un agente basado en LLM capaz de recibir entradas en lenguaje natural, analizarlas y transformarlas en decisiones concretas, esto a través del ofrecimiento de opciones según disponibilidad, preferencias y restricciones temporales de los usuarios, con el fin de facilitar la coordinación de actividades grupales.

2.3.2. Objetivos específicos

1. Para el 8 de mayo, diseñar y documentar la arquitectura del sistema, incluyendo el componente de persistencia (archivos JSON), la integración con el LLM, los diagramas UML (casos de uso, secuencia, componentes) y la especificación de endpoints de la API, validando que cubra los requisitos de utilidad para el usuario a través de una minidemo funcional.
2. Antes del 22 de mayo, implementar un MVP parcial ejecutable en entorno local, que permita el flujo completo de conversación → extracción de disponibilidad → revisión manual → cálculo de horarios sugeridos → confirmación, con un manejo básico de errores y conexión funcional entre frontend y backend.
3. Hasta el 7 de julio, diseñar e implementar mecanismos de guardrails (filtros de entrada, restricción temática, validación de salida) que limiten el comportamiento del LLM al dominio de coordinación, logrando reducir las alucinaciones y rechazar prompts maliciosos en al menos el 90 % de los casos de prueba documentados.
4. Desde el 23 de mayo al 23 de junio, evaluar el sistema en un entorno real mediante al menos 3 casos de uso con usuarios simulados, midiendo la precisión de la decisión propuesta (objetivo $\geq 85\%$) y la satisfacción percibida (encuesta con escala 1-5, objetivo ≥ 4), entregando un informe de resultados para la evaluación final.

3. Estudio de factibilidad

3.1. Análisis técnico

e

3.2. Análisis económico

f

3.3. Evaluación de riesgos y limitaciones

g

4. Metodología de desarrollo

4.1. Selección de una metodología ágil

Se a decidido que para el desarrollo de este proyecto se utilizará la metodología Scrum, esta metodología altamente conocida por su enfoque iterativo incremental promueve la adaptabilidad y la entrega continua, de esto se desligan distintas razones de porque fue seleccionada:

1. **Iteraciones cortas (Sprints):** Los ciclos cortos de Scrum son la clave para este proyecto, el poder contar con versiones funcionales desde etapas tempranas permitirá medir el avance y el valor que esta entregando la solución casi desde el principio, y así evaluar la viabilidad de posibles incrementos.
2. **Retroalimentación temprana:** Siguiendo con lo del punto anterior, estas versiones funcionales casi al inicio del ciclo de vida del proyecto permitirá realizar pruebas que permitan detectar de forma oportuna errores o desviaciones, mejorando la calidad del producto final.

3. **Alta adaptabilidad a cambios:** La posibilidad mencionada anteriormente de recibir retroalimentación continua sobre el producto permitirá adaptarse a cualquier cambio que sea necesario que fuera imprevisto durante la planificación inicial. Esto es clave teniendo en cuenta que es el primer proyecto basado en LLM de los integrantes del grupo, por lo que se podrían pasar cosas por alto en una planificación inicial por mera falta de experiencia en el área.

4.2. Planificación de iteraciones y definición de roles en el equipo

4.2.1. Plan de iteraciones

La planificación se realiza considerando las fechas de evaluación oficiales del curso. Se tomó en cuenta el estado actual del proyecto junto con el estado objetivo. Las primeras iteraciones se enfocan en consolidar el estado actual y realizar diferentes pruebas, mientras que los siguientes Sprints se enfocan en completar los elementos faltantes del MVP.

Sprint	Fechas	Objetivo principal	Entregable esperado
Sprint 0: For- mulación y pro- totipo inicial	05/05 al 08/05	Definir la base conceptual, técnica y metodológica del proyecto.	Informe de Evaluación 1, presentación, repositorio y mini-demo funcional.
Sprint 1: Con- solidación del flujo principal	09/05 al 22/05	Estabilizar el flujo con- versación, extracción, revi- sión, cálculo y confirma- ción.	MVP parcial ejecutable lo- calmente con flujo princi- pal estable.
Sprint 2: Ca- nal simulado y avance funcio- nal del MVP	23/05 al 09/06	Preparar un avance demos- trable equivalente al me- nos al 50 % del MVP, usan- do entrada conversacional y canal simulado.	Demo de Evaluación 2 con extracción, decisión sugerida, confirmación y simula- ción de canal.
Sprint 3: Inte- gración con ca- nal de comuni- cación y calen- dario	10/06 al 23/06	Implementar al menos una integración funcional con un canal de comunicacio- nes y una integración mí- nima con calendario.	Canal externo mínimo co- nectado al backend y deci- sión confirmada exportable a calendario.
Sprint 4: Guar- drails, privaci- dad y cierre del MVP	24/06 al 07/07	Completar el MVP com- prometido, reforzar seguri- dad y preparar evidencias finales.	Informe final, presenta- ción, temario de demo, tra- zabilidad de funcionalida- des y MVP completo.
Buffer de defen- sa y respaldo	08/07 al 09/07	Reducir riesgos técnicos durante la presentación fi- nal.	Demo final ensayada, res- paldada y preparada para casos de falla.

Tabla 1: Planificación general de iteraciones del proyecto mediante Scrum.

De forma complementaria, las actividades específicas de cada Sprint se organizan de la siguiente manera:

- **Sprint 0:** definición del problema, propósito comercial, objetivos, factibilidad, arquitectura inicial, roles Scrum, repositorio, demo mínima del LLM y planificación metodológica.
- **Sprint 1:** revisión de endpoints, validación de esquemas de datos, ajuste de prompts, pruebas de extracción con LLM o fallback, mejora del manejo de errores, validación de corrección manual y conexión frontend-backend.
- **Sprint 2:** preparación de un caso de uso realista, carga de conversación ejemplo, extracción de disponibilidades, visualización de participantes y datos faltantes, cálculo de mejores horarios, explicación de cobertura, confirmación de una opción y uso del canal simulado tipo mensajería grupal.
- **Sprint 3:** implementación de al menos una integración funcional con un canal de comunicaciones. Para el MVP se prioriza una integración mediante bot de Discord, debido a que permite recibir mensajes desde un canal y reenviarlos al backend. Además, se implementará una integración mínima con calendario mediante generación de evento en formato `.ics`, permitiendo que la decisión confirmada sea importada en aplicaciones como Google Calendar, Outlook o Apple Calendar.
- **Sprint 4:** pruebas con casos borde, entradas ambiguas, horarios incompatibles, participantes sin disponibilidad, errores del LLM, posibles prompt injections, documentación de privacidad, trazabilidad de funcionalidades, capturas, snippets y arquitectura final.
- **Buffer:** ensayo de demo, preparación de datos precargados, capturas alternativas, respaldo del evento de calendario, script de presentación y preparación de respuestas ante preguntas del profesor.

4.3. Diagrama de usos éticos de la IA (MyAI Compass)

a

4.4. Repositorio de control de versiones

Para el control de versiones del código y los informes del proyecto se hará uso de github, específicamente del repositorio presente en el siguiente enlace: <https://github.com/Ch3chS/TAVI-Charlie>.

5. Diseño arquitectural

5.1. Definición de la arquitectura de software

La solución que se ha implementado cuenta con un patrón cliente web + BFF + servicios internos. El frontend, desarrollado en React consume rutas del backend FastAPI. El backend actua como BFF se encarga de la extracción del LLM, la fusión de los datos dentro de la sesión, el cálculo de opciones, la persistencia y las respuestas para la interfaz de usuario (canal). Para la creación de la aplicación el archivo `main.py` se encarga del CORS; `routes.py` concentra las interfaces; `session_service.py` se encarga de las sesiones; `decision_engine.py` se encarga de la toma de decisión sobre la mejor sesión; `json_repository.py` se encarga de la persistencia de los datos.

Además, se considera una capa de integración para canales de comunicación. En el estado actual del proyecto, el sistema cuenta con un canal simulado para las conversaciones grupales. Para el desarrollo del MVP se contempla la implementación de al menos una integración real, priorizando Discord debido a su simplicidad de integración. La capa quedara diseñada para permitir futuras integraciones ya sea con WhatsApp, Slack u otras plataformas. Finalmente, se incorporará un servicio de calendario que permitira consolidar e integrar la decisión confirmada en un evento calendarizable, inicialmente en un formato `.ics`.

El flujo general quedaria asi: mensaje del usuario → canal web o externo → BFF → servicio LLM extractor → actualización de sesión → motor de decisión →

respuesta al usuario → confirmación → generación de evento de calendario.

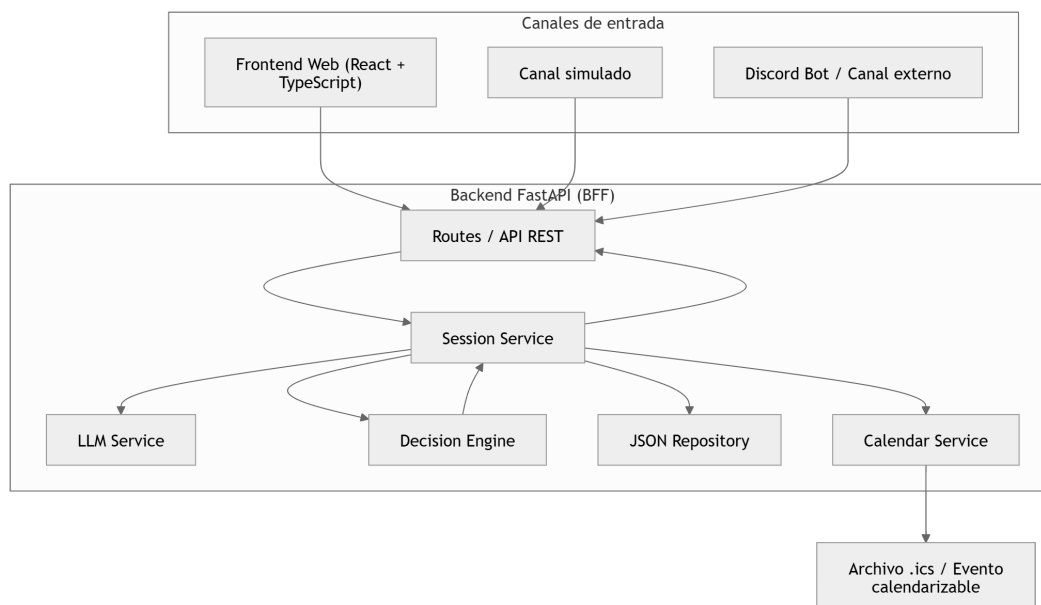


Figura 1: Diagrama Arquitectura general

5.2. Identificación de la integración del LLM en la solución

El LLM se integra como un componente dentro del servicio *LLM Service*. Es importante que la función no es tomar una decisión final, sino transformar los mensajes en lenguaje natural a datos estructurados en un formato específico para que puedan ser utilizados el resto del sistema.

Por ejemplo, si llegase un mensaje como “Nicolás puede el lunes en la tarde y Matías puede desde las 16”, el LLM debe extraer participantes, días y horarios disponibles. Esta información es devuelta en un formato estructurado en JSON, para que el backend lo procese y lo fusione con el resto de la información de la sesión.

La decisión final depende del servicio *Decision Engine*, el servicio se encarga de calcular de manera determinística la mejor opción evaluando la cobertura de los participantes y generando una explicación para cada alternativa generada.

La integración considera mecanismos básicos de seguridad como prompts restrictivos, validación de la salida, corrección manual de los datos reconocidos por el

sistema, uso de fallback en caso de fallas del modelo y la confirmación explícita del usuario a la hora de la toma de decisión final.

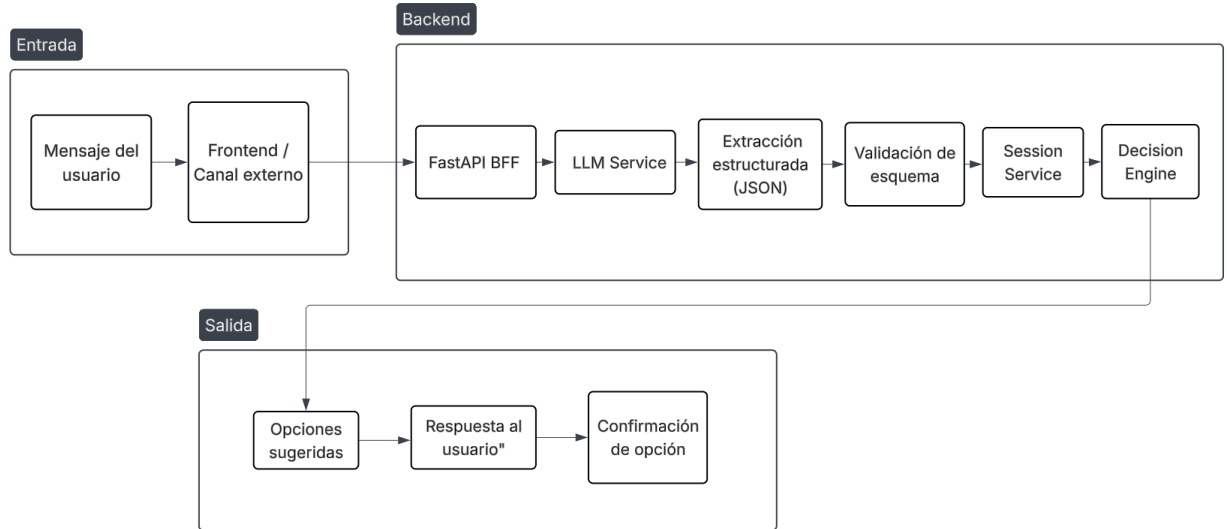


Figura 2: Flujo de integración del LLM en el proceso de coordinación.

5.3. Diagramas UML y especificaciones técnicas

Para el diseño del sistema se consideran tres vistas principales: una vista de componentes, una vista de flujo y una vista de clases. La vista de componentes permite observar la separación entre frontend, backend, servicios internos e integraciones externas. La vista de flujo permite describir cómo un mensaje en lenguaje natural se transforma en una decisión grupal. Finalmente, la vista de clases permite representar las entidades principales utilizadas por el sistema.

Las clases principales del modelo son **Session**, **Participant**, **TimeSlot**, **TimeOption**, **AvailabilityCell**, **ChannelMessage** y **ChannelConfig**. La clase central es **Session**, ya que representa una coordinación específica. Esta contiene participantes, mensajes, disponibilidades, opciones calculadas, matriz de disponibilidad y, en caso de confirmación, una opción seleccionada y un resumen final de la decisión.

En este diseño, **Participant** solo corresponde a una persona detectada en una sesión, no a una cuenta global. Si se requiere en un futuro debería crearse la clase

User para soportar esto.

A nivel técnico, el MVP utiliza React y TypeScript para el frontend, FastAPI y Python para el backend, comunicación mediante HTTP/REST, persistencia local en JSON, extracción mediante LLM configurable y un motor determinístico para el cálculo de opciones. Además, se contempla una integración mínima con un canal de comunicación externo y una integración con calendario mediante generación de archivo .ics.

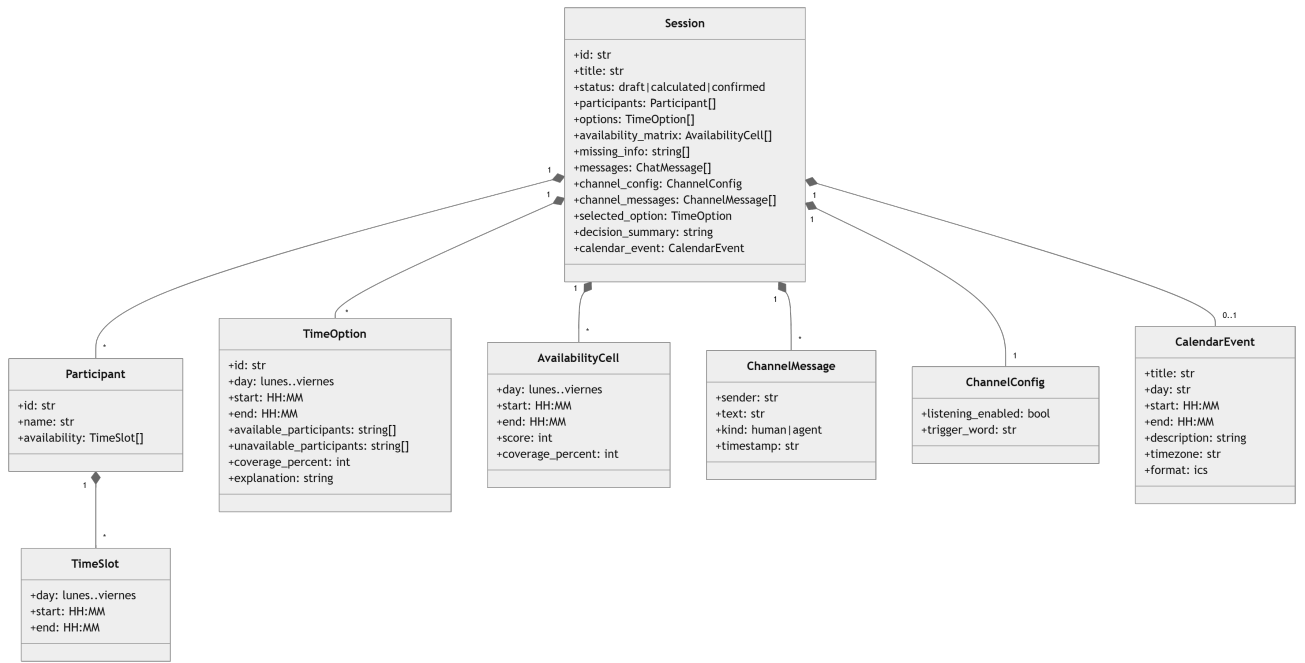


Figura 3: Diagrama de clases principales del sistema.

6. Mini-demo funcional

6.1. Despliegue de una instancia mínima de un LLM

Para comprobar la factibilidad técnica del proyecto se ha implementado una minidemo funcional la cuál presenta la estructura básica del proyecto en un entorno local controlado. La implementación realizada demuestra que el agente utilizado es capaz de procesar mensajes en lenguaje natural y ofrecer alternativas de horarios para

realizar actividades grupales.

6.1.1. LLM utilizado y justificación

Ya que el LLM es el núcleo del presente proyecto se analizaron unas cuantas opciones tanto por si se tenía que ejecutar en local o en la nube, específicamente para la minidemo, al ser algo inicial y para mantener cierto nivel de seguridad se optó por usar un modelo corriendo en local, específicamente el modelo **Qwen3-4B-Instruct-2507-Q4-K-M**, el cual cuenta, como dice su nombre con aproximadamente 4B (4 mil millones) de parámetros alcanzando aproximadamente 2.5 GB.

Este modelo que para una máquina local es relativamente grande, pero fue elegido por:

1. Comprende mejor el lenguaje natural, lo cuál es relevante ya que utilizará mensajes reales.
2. Es capaz de seguir el formato JSON que se utiliza para la persistencia.
3. Es mejor que uno pequeño detectando participantes y restricciones.
4. Puede manejar de mejor manera frases mezcladas.
5. Es relativamente menos propenso a alucinar.

Algunos contras que tiene el tamaño de este modelo son:

1. Al tener más parámetros suele ser más lento.
2. Consume más RAM.

A pesar de dichas limitaciones sigue siendo factible en los dispositivos del equipo Charlie, por lo que los pros mencionados anteriormente parecen suficientes para justificar esta elección.

6.1.2. Prompt de personalización inicial

Para extraer la información de los distintos usuarios en un formato utilizable se realizó el siguiente prompt de sistema:

```
'''
```

```
Extrae disponibilidad desde el mensaje del usuario.
```

```
Devuelve SOLO JSON con esta forma:
```

```
{
  "participants": [
    {
      "name": "string",
      "availability": [
        {
          "day": "lunes|martes|miercoles|jueves|viernes",
          "start": "HH:MM",
          "end": "HH:MM"
        }
      ]
    }
  ]
}
```

```
Reglas:
```

- No inventes participantes.
- No inventes dias si no aparecen.
- Si dice "manana", usa 09:00-12:00.
- Si dice "tarde", usa 15:00-18:00.
- Si dice "desde las 16", usa 16:00-18:00.
- Si dice "cualquier hora", usa 09:00-18:00.
- Si dice "todos los dias", crea disponibilidad para lunes, martes, miercoles, jueves y viernes.
- Si falta horario, deja availability vacio.
- Responde solo JSON, sin markdown.

```
'''
```

6.1.3. Presentación y exposición

Finalmente se realizó el entorno que puede ser visto en la sección anterior con la arquitectura. Se intentó realizar una interfaz de usuario relativamente entendible y algunos scripts con el fin de que sea fácil revisar pruebas que demuestren la funcionalidad de la minidemo para recibir un feedback lo más completo posible, para de esta manera poder realizar las mejoras correspondientes para la siguiente entrega.

7. Conclusiones

En esta primera entrega se ha establecido la estructura base del proyecto. A partir de la identificación de una problemática real y frecuente en entornos organizacionales (la ineficiencia en la coordinación grupal) se definió una solución centrada en un agente con un LLM como núcleo. El estudio de factibilidad evidencia que el proyecto es técnica y económicamente realizable dentro del plazo de un semestre, con un stack tecnológico relativamente accesible. La adopción de Scrum como metodología de desarrollo, junto con la definición clara de roles y un diseño arquitectural modular, sienta las condiciones para un avance iterativo y controlado hacia el MVP. Las siguientes etapas del proyecto se enfocarán en la implementación progresiva de los sprints definidos, con miras a demostrar en la Evaluación 2 un avance funcional de al menos el 50 % de los objetivos comprometidos, tal como se pide.